



KeyDB

von Lasse Tristan Feldermann Welkner



Inhalt

1. Was ist KeyDB? Worauf basiert KeyDB? Wofür wird KeyDB genutzt?
2. KeyDB CLI: Basics
 - 2.1. Erstellen von Objekten & Datatypes
 - 2.2. Testprojekt zum Ausprobieren & Aufgabe 1
3. Konfiguration
4. Tutorial: Anbindung an simple CRUD Applikation & Aufgabe 2
5. Mass Insertion via Pipemode & KeyDB protocol Format
6. Skalierung: KeyDB Cluster
7. Partitionierung nochmal researchen ob ich den punkt noch brauch
8. Fazit



Was ist KeyDB

- high-performance, multi-threaded, in-memory NoSQL database
- Persistenz möglich
- basiert auf Redis
- verwirft sekundäre Datenbankmodelle
- Primäres Datenbankmodell Key-Value Store



KeyDB Data Types

Strings:

- Beschreibung: Einfache Schlüssel-Wert-Paare, die Text oder Binärdaten enthalten können.
- Beispiel: SET key "Hello"

Hashes:

- Beschreibung: Assoziative Datenstrukturen, die Felder und zugehörige Werte speichern.
- Beispiel: HSET user:id name "John"

Listen:

- Beschreibung: Geordnete Sammlungen von Elementen, die am Anfang oder Ende erweitert werden können.
- Beispiel: LPUSH queue "Task 1"



KeyDB CLI

- INFO: Gibt umfassende Informationen zur KeyDB-Instanz zurück, einschließlich Speichernutzung, Clients, Latenzen usw
- FLUSHDB: Löscht alle Schlüssel in der aktuellen Datenbank.
- FLUSHALL: Löscht alle Schlüssel in allen Datenbanken der KeyDB-Instanz.
- GET key: Liest den Wert eines Schlüssels.
- SET key value: Setzt den Wert eines Schlüssels auf einen bestimmten Wert.
- DEL key: Löscht einen bestimmten Schlüssel und seinen Wert.
- HSET und HGET: Arbeiten mit Hashes
- LPUSH und LRANGE: Arbeiten mit Listen



Setup Testprojekt

https://github.com/speedprogrammer9000/Big_Data_DEMO

Starten der Services: `docker-compose up --build`

Attachen an KeyDB Instanz: `docker exec -it <container_id> /bin/sh`



Konfiguration

key	value
dbfilename	<filename>
dir	<abs/path/to/workingdir>
save	<integer> <integer>
logfile	<filename>
replicaof	<ip> <port>



Aufgabe 2

siehe Repository :)



Mass Insertion via Pipemode & KeyDB protocol Format

- Der Pipemode erlaubt das Laden großer Datenmengen durch die Verarbeitung von Befehlen in einer Batch-Datei.
- Dies ist besonders effizient, da die Daten ohne aufwendige Client-Server-Kommunikation in einem einzigen Prozess verarbeitet werden können.
- Das KeyDB-Protokollformat optimiert die Kommunikation zwischen Clients und der Datenbank, indem es eine effiziente Binärrepräsentation verwendet.
- Der Einsatz von Pipemode und KeyDB-Protokollformat verbessert die Geschwindigkeit und Effizienz bei Masseninsertionen im Vergleich zu herkömmlichen Methoden.



Mass Insertion via Pipemode & KeyDB protocol Format

data.txt:

```
*3\r\n$3\r\nSET\r\n$5\r\nuser1\r\n$5\r\nJohn\r\n\r\n
```

```
*3\r\n$3\r\nSET\r\n$5\r\nuser2\r\n$6\r\nAlice\r\n\r\n
```

```
*3\r\n$3\r\nSET\r\n$5\r\nuser3\r\n$4\r\nBob\r\n\r\n
```

```
*3\r\n$3\r\nSET\r\n$5\r\nuser4\r\n$7\r\nCharlie\r\n\r\n
```

bash

```
cat data.txt | keydb-cli --pipe
```



Skalierung mit KeyDB Cluster

- Der KeyDB Cluster erlaubt die horizontale Skalierung durch Verteilung von Daten und Last auf mehrere Server.
- Die Datenbankinstanzen arbeiten zusammen, um eine hohe Verfügbarkeit zu gewährleisten und Lasten zu bewältigen.
- Automatische Partitionsverwaltung erleichtert die Verwaltung und Skalierung des Clusters.
- Durch die horizontale Skalierung werden sowohl Lese- als auch Schreiblasten besser verteilt, was die Gesamtleistung verbessert.



Skalierung mit KeyDB Cluster

Ansätze zur automatischen Partitionierung von Daten:

Hash-basierte Partitionierung:

- Ein häufig verwendetes Verfahren ist die hashbasierte Partitionierung, bei der ein Hash-Wert des Schlüssels berechnet wird. Basierend auf diesem Hash-Wert wird entschieden, in welcher Partition die Daten gespeichert werden.

Range-basierte Partitionierung:

- Eine andere Möglichkeit ist die rangebasierte Partitionierung, bei der Daten in bestimmten Bereichen (Ranges) gespeichert werden. Dies könnte auf numerischen oder alphanumerischen Ranges basieren. Zum Beispiel könnten Daten mit Schlüsseln im Bereich von A-M auf einer Partition und N-Z auf einer anderen Partition gespeichert werden.

List-basierte Partitionierung:

- Hier werden Daten basierend auf einer vordefinierten Liste von Partitionen zugeordnet. Jede Partition enthält eine spezifische Menge von Schlüsseln, die aufgrund von benutzerdefinierten Kriterien ausgewählt wurden.



9. Fazit

- Hohe Leistungsfähigkeit
- Multi-Threaded-Architektur
- Automatische Skalierung durch KeyDB Cluster
- Vielseitigkeit
- ease-of-use



Quellen

<https://docs.keydb.dev/>

<https://db-engines.com/de/system/KeyDB%3BRedis>